

Computer Architecture

Course No. 21472

Week #4

Arithmetic for Computers I

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430

Retrospect: Procedure Calling

Steps Required

1. Place parameters in Registers
2. Transfer control to procedure
3. Acquire Storage for procedure
4. Perform Procedure's operations
5. Place result in Register for caller
6. Return to place of call

PC : program counter

Register containing the address of the instruction in the program being executed

$ra : return address

Register containing the return address of a procedure

Procedure Call

```
jal ProcedureLabel
```

- jump-and-link
- Address of following instruction (**PC+4**) put in **$ra**
- Jumps to target address

Procedure Return

```
jr $ra
```

- jump-register
- Copies **$ra** to program counter (PC)
- Can also be used for computed jumps

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retrospect: Register Usage

- **$a0 - $a3** : arguments (reg's 4-7)
- **$v0, $v1** : result values (reg's 2 and 3)
- **$t0 - $t9** : temporaries (NOT preserved)
- **$s0 - $s7** : saved (saved/restored by callee)
- **$gp** : global pointer for static data (reg 28)
- **$sp** : stack pointer (reg 29)
- **$fp** : frame pointer (reg 30)
- **$ra** : return address (reg 31)

Stack

- A data structure for spilling registers organized as a **last-in-first-out** (LIFO) queue
- **Stack Pointer** ($sp) : A register for most recently allocated address in a stack
- Stack grows "down" : from high memory address to low memory address

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retrospect: Leaf Procedure Example

- Example C code:

```
int leaf_example (int g, h, i, j)
{
    int f;
    f = ( g + h ) - ( i + j );
    return f;
}
```

- Arguments g, …, j in $a0, …, $a3
- f in $s0 (hence, need to save $s0 on stack)
- Result in $v0

```
leaf_example:
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retrospect: Non-Leaf Procedure Example

- Example C code:

```
int fact (int n)
{
    if (n < 1) return f;
    else return n * fact(n - 1);
}
```

- Argument n in $a0
- Result in $v0

```
fact:
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Arithmetic for Computers

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Floating Point Real Numbers

- Representation and Operation

Precision

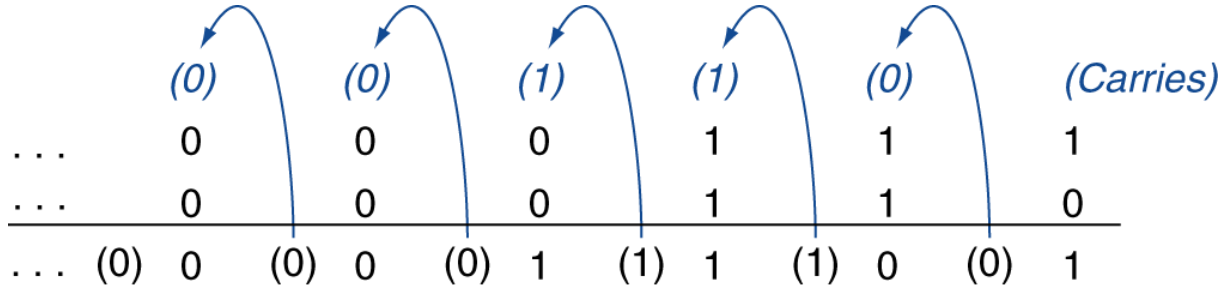
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Integer Addition

Example : 7 + 6 = ?



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Integer Subtraction

Example : 7 - 6 = ?

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Overflow if result is out of range (e.g., 32 bits)

Overflow in Integer Addition

- Adding positive and negative operands: None
- Adding two positive operands: Overflow if result sign is 1
- Adding two negative operands: Overflow if result sign is 0

Overflow in Integer Subtraction

- Subtracting two positive operands: None
- Subtracting positive from negative operands: Overflow if result sign is 0
- Subtracting negative from positive operands: Overflow if result sign is 1

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Overflow if result is out of range (e.g., 32 bits)

Overflow in Integer Addition

- Adding positive and negative operands: None
- Adding two positive operands: Overflow if result sign is 1
- Adding two negative operands: Overflow if result sign is 0

Overflow in Integer Subtraction

- Subtracting two positive operands: None
- Subtracting positive from negative operands: Overflow if result sign is 0
- Subtracting negative from positive operands: Overflow if result sign is 1

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Overflow if result is out of range (e.g., 32 bits)

Overflow in Integer Addition

- Adding positive and negative operands: None
- Adding two positive operands: Overflow if result sign is 1
- Adding two negative operands: Overflow if result sign is 0

Overflow in Integer Subtraction

- Subtracting two positive operands: None
- Subtracting positive from negative operands: Overflow if result sign is 0
- Subtracting negative from positive operands: Overflow if result sign is 1

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Overflow if result is out of range (e.g., 32 bits)

Overflow in Integer Addition

- Adding positive and negative operands: None
- Adding two positive operands: Overflow if result sign is 1
- Adding two negative operands: Overflow if result sign is 0

Overflow in Integer Subtraction

- Subtracting two positive operands: None
- Subtracting positive from negative operands: Overflow if result sign is 0
- Subtracting negative from positive operands: Overflow if result sign is 1

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Overflow if result is out of range (e.g., 32 bits)

Overflow in Integer Addition

- Adding positive and negative operands: None
- Adding two positive operands: Overflow if result sign is 1
- Adding two negative operands: Overflow if result sign is 0

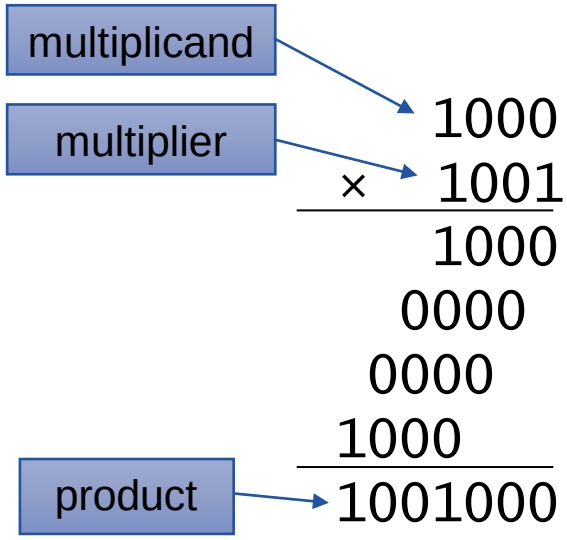
Overflow in Integer Subtraction

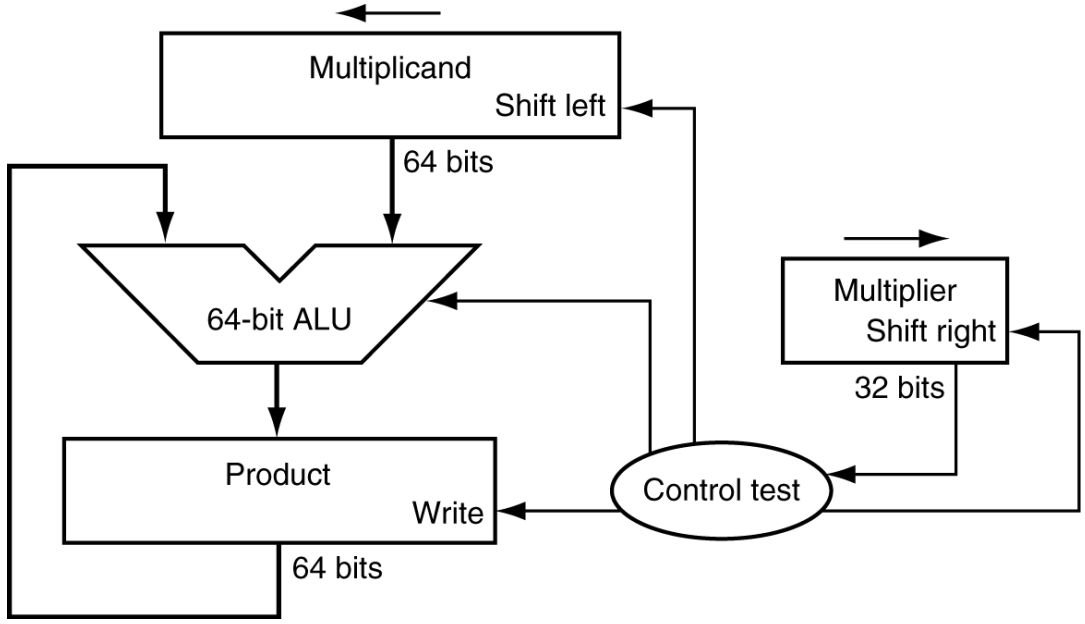
- Subtracting two positive operands: None
- Subtracting positive from negative operands: Overflow if result sign is 0
- Subtracting negative from positive operands: Overflow if result sign is 1

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

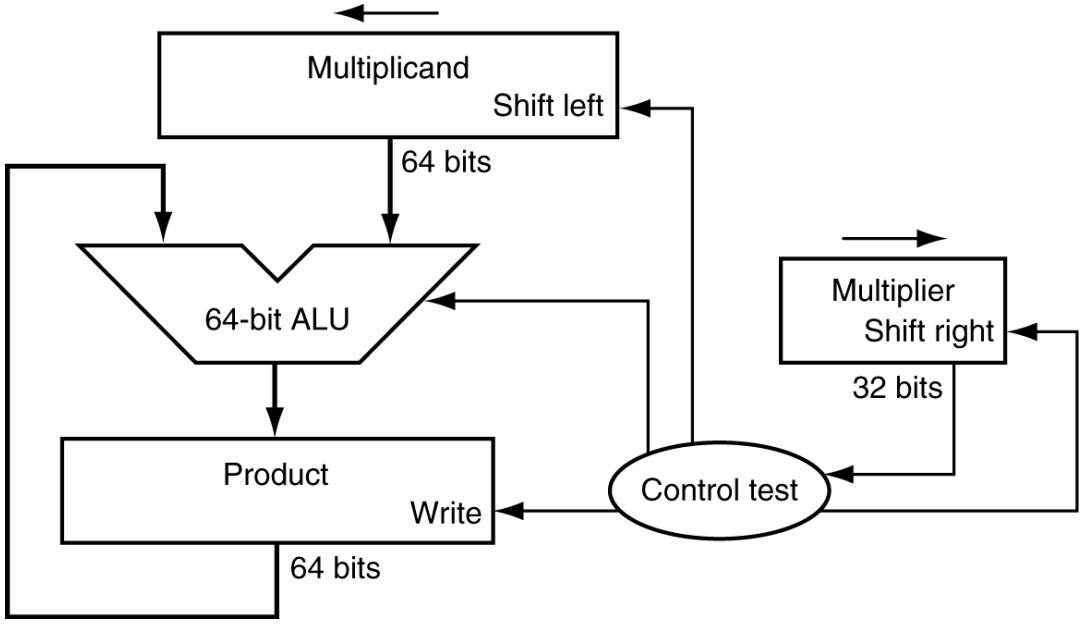


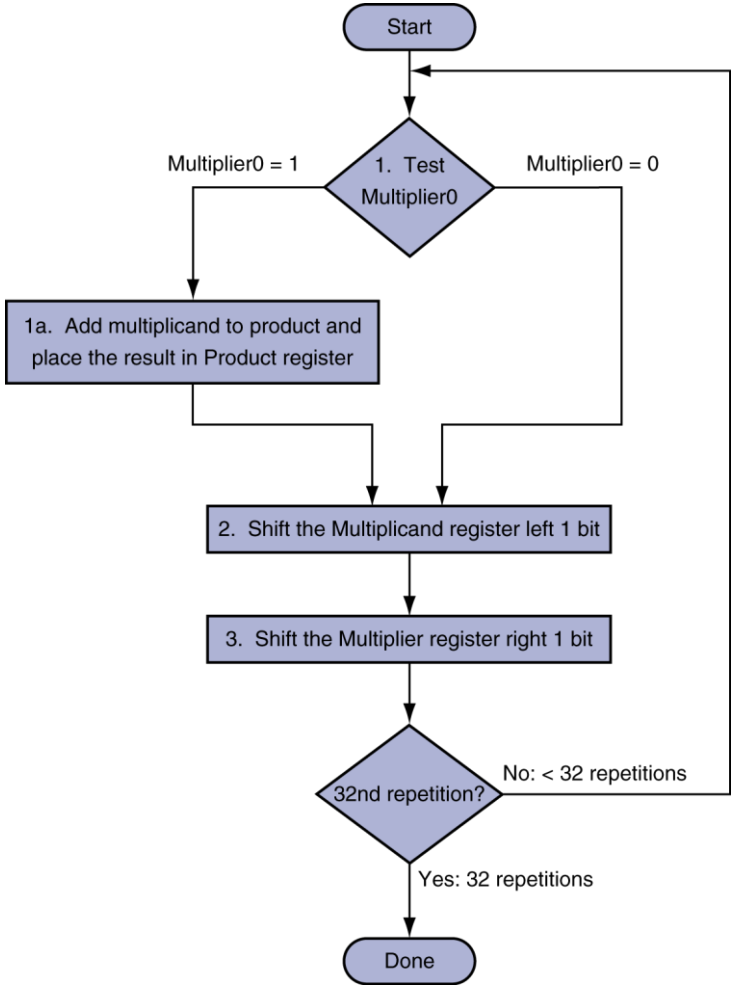


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

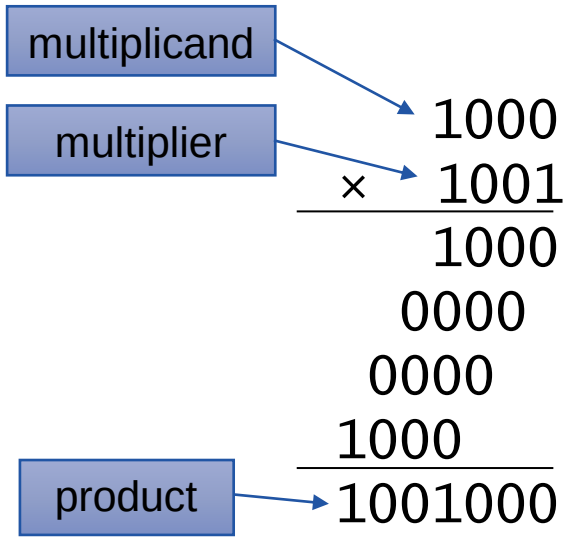


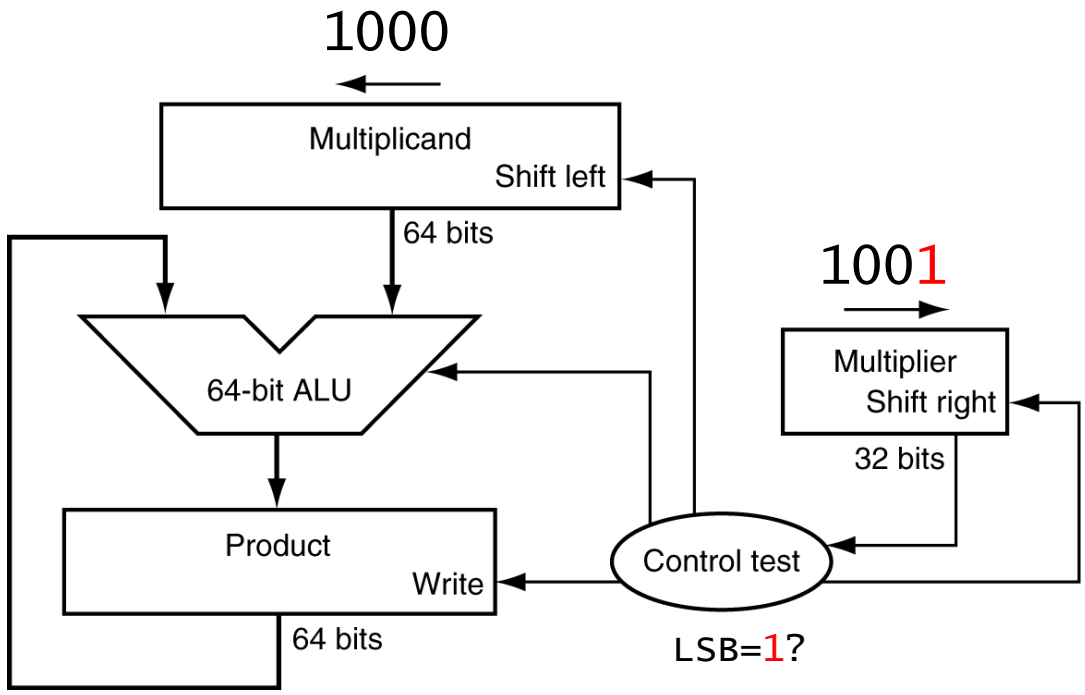


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

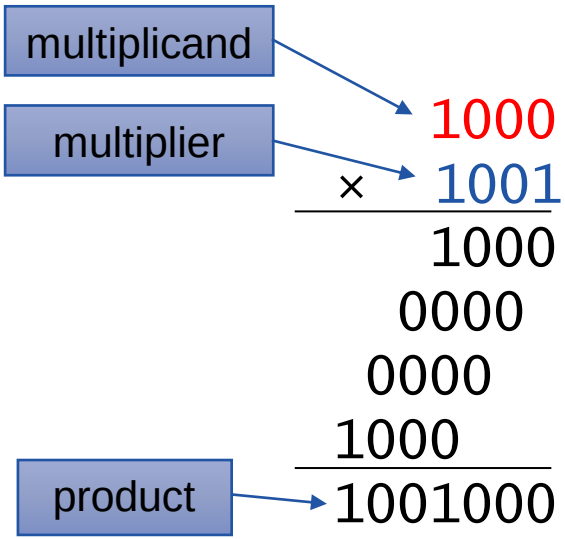


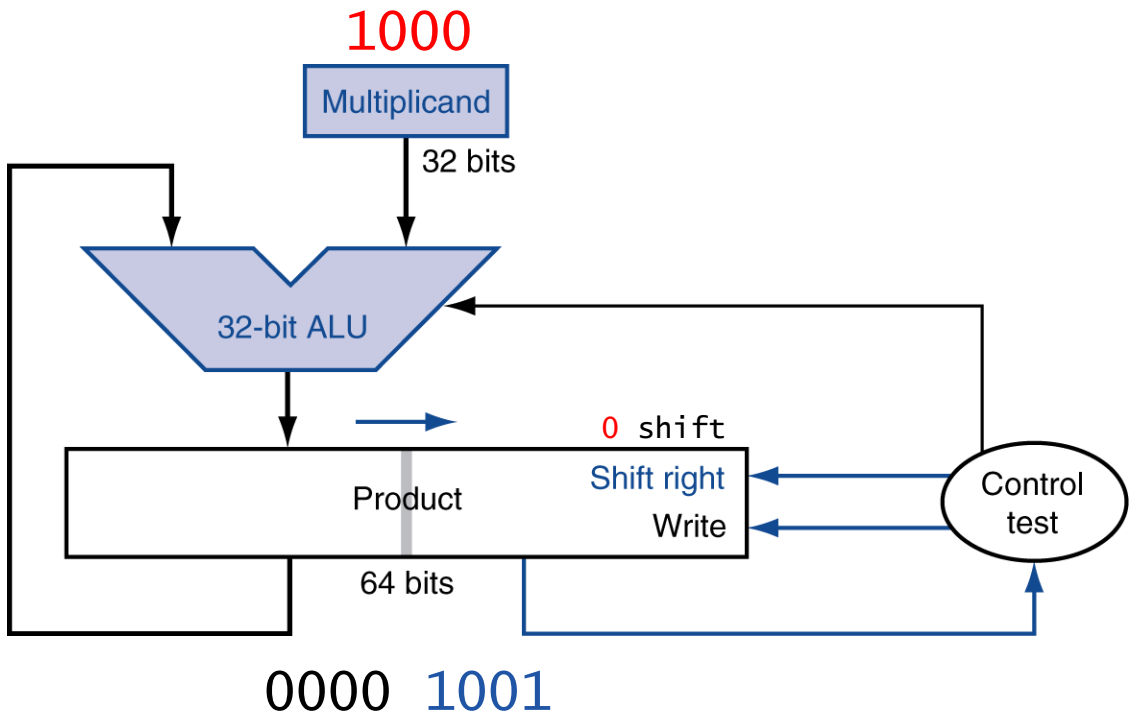


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow





Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

- Two 32-bit registers for product
  - HI: most-significant 32 bits
  - LO: least-significant 32-bits

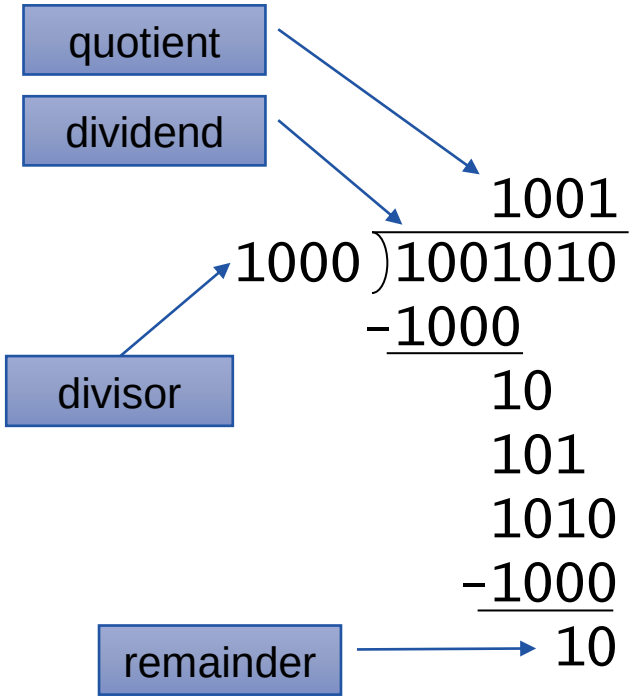
- Instructions
  - `mult rs, rt` / `multu rs, rt`
    - 64-bit product in HI/LO
  - `mfhi rd` / `mflo rd`
    - Move from HI/LO to rd
    - Can test HI value to see if product overflows 32 bits
  - `mul rd, rs, rt`
    - Least-significant 32 bits of product –> rd

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Arithmetic for Computers

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow



n-bit operands yield *n*-bit quotient and remainder

Dividend = Quotient × Divisor + Remainder

Check for 0 divisor

Long division approach

- If divisor ≤ dividend bits
  - 1 bit in quotient, subtract
- Otherwise
  - 0 bit in quotient, bring down next dividend bit

Restoring division

- Do the subtract, and if remainder goes < 0, add divisor back

Signed division

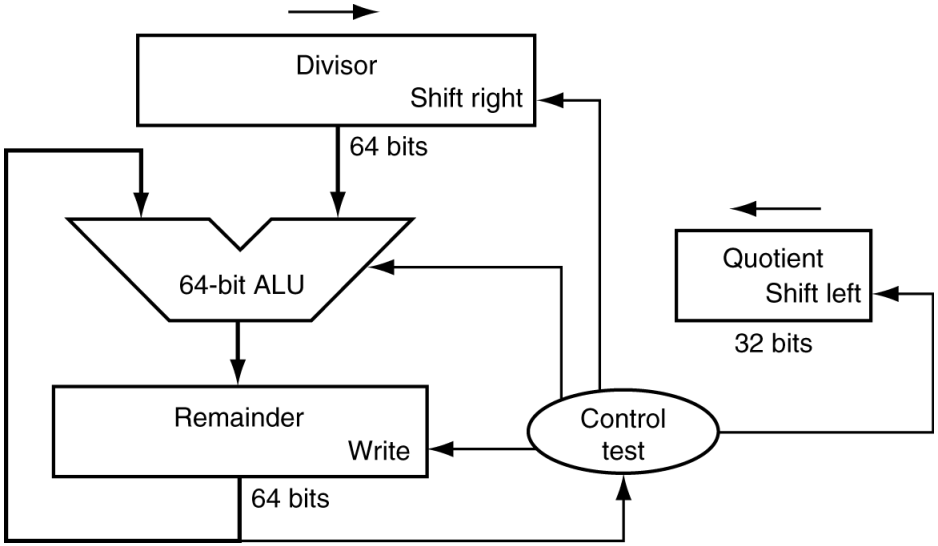
- Divide using absolute values
- Adjust sign of quotient and remainder as required
- Dividend and remainder must have the same signs

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

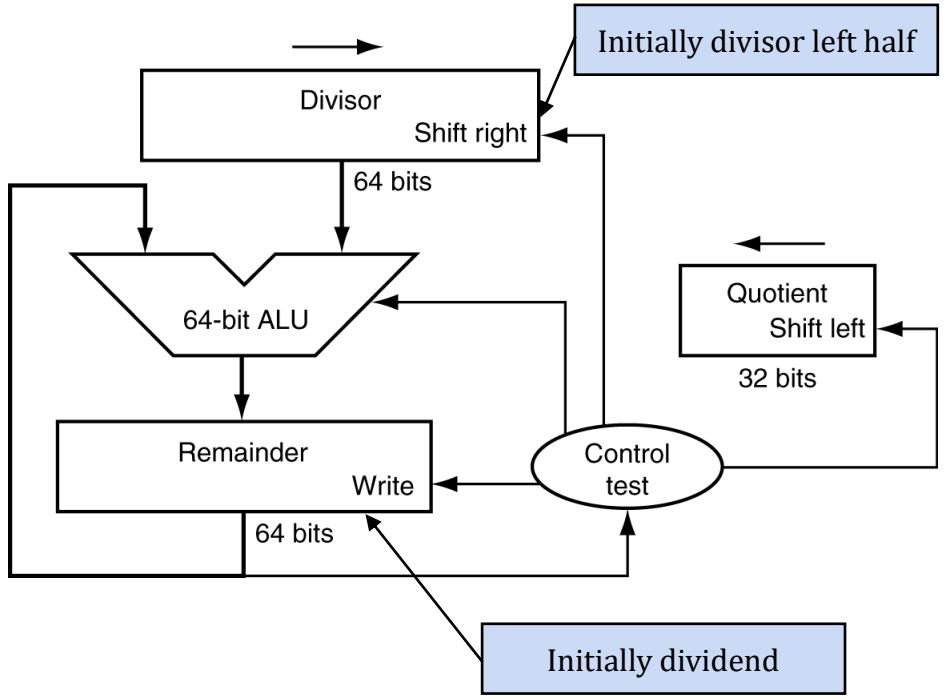
```
    01001
1000)01001010
```



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential



Thank you

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430